

Olist Database Engineering Report

Import, Optimization, and Query Performance

Victoria Johnson | MySQL, Python (pandas, SQLAlchemy)

Project Overview

This report documents the database preparation work completed before analyzing the Olist Brazilian E-Commerce dataset. During the import process, incorrect column types prevented index creation and contributed to slow multi-table queries. I corrected the schema, created indexes on frequently joined columns, and tested the changes by rerunning a query that had previously failed.

Initial State. What Was Wrong

The Import Problem

During the initial import from pandas to MySQL through SQLAlchemy, several string columns were created as TEXT and date columns were not stored as DATETIME. This created three problems:

- ID columns like `customer_id`, `order_id`, and `product_id` were imported as TEXT instead of `VARCHAR(50)`.
- Date columns like `order_purchase_timestamp` and `review_creation_date` were imported as TEXT instead of DATETIME.
- MySQL cannot create indexes on TEXT columns without a specified key length, which made JOIN operations across large tables extremely slow.

The Errors

Error	What It Meant
Error 1170	BLOB/TEXT column used in key specification without a key length. MySQL refused to index the column.
Error 2013	Lost connection to MySQL server during query. Occurred while running a large multi-table JOIN before the schema and indexing changes were applied..

Optimization Work

1. Data Type Corrections

All tables were re-imported with explicit data type mappings passed to the `dtype` parameter in `pandas.to_sql()`:

Table	Column	Changed To
customers	<code>customer_id</code> , <code>customer_unique_id</code>	<code>VARCHAR(50)</code>
orders	<code>order_id</code> , <code>customer_id</code>	<code>VARCHAR(50)</code>
orders	<code>order_purchase_timestamp</code> , <code>order_approved_at</code> , <code>order_delivered_customer_date</code> , <code>order_estimated_delivery_date</code>	DATETIME

order_items	order_id, product_id, seller_id	VARCHAR(50)
order_items	price, freight_value	Float()
order_reviews	review_id, order_id	VARCHAR(50)
order_reviews	review_score	Integer()
products	product_id, product_category_name	VARCHAR(50/100)

2. Index Creation

Created indexes on key columns used for joins and filtering across the analytical tables: customers: customer_id, customer_unique_id

- orders: customer_id, order_id, order_status
- order_items: order_id, product_id, seller_id
- products: product_id, product_category_name
- category_translation: product_category_name

3. Verified Fix

The category analysis query had previously failed with a lost-connection error. After correcting the column types and creating the indexes, the same analysis completed in 8.13 seconds. The query joined five tables and processed the Olist analytical dataset, providing a practical check that the database changes improved query execution.

Why TEXT Columns Break Indexing

MySQL indexes allow the database engine to locate matching values more efficiently instead of relying on full table scans for every lookup.

In this project, several identifier columns were imported as TEXT. MySQL requires a prefix length when indexing TEXT columns, which prevented the standard indexes I attempted to create and produced Error 1170.

The identifier values had known, limited lengths, so VARCHAR was a more appropriate data type for columns such as customer_id and order_id. After changing these columns to VARCHAR(50), the indexes were created successfully. The previously failing category analysis then completed in 8.13 seconds.

VARCHAR columns do have a defined maximum length (e.g. VARCHAR(50) means no more than 50 characters). MySQL can index these directly. Once the customer_id and order_id columns were changed from TEXT to VARCHAR(50), the indexes created successfully and JOIN performance improved from timeout to single-digit seconds.

Technical Skills in This Work

Area	What Was Done
ETL	Imported 8 CSV files, defined data types in Python, and loaded the tables into MySQL with an explicit schema.
Schema Design	Defined explicit VARCHAR, Float, Integer, and DATETIME types for each column across all 8 tables.
Indexing	Created 14 analytical indexes across all JOIN columns to support query performance.
Error Diagnosis	Investigated Error 1170 and Error 2013 during database preparation. Identified the TEXT data type as the indexing issue, changed identifier columns to VARCHAR, and reran the previously failing query after the schema and indexing changes.
Query Optimization	Improved execution of a five-table analysis query that had previously failed with a lost-connection error. After the schema and indexing changes, the query completed in 8.13 seconds.
Python Scripting	Wrote a reusable Python import script using pandas and SQLAlchemy with parameterized dtype maps and automated index creation.

Contact

Victoria Johnson

Business and Data Analyst | SQL, Power BI, Python, Excel

Email: Johnsonfisayo090@gmail.com

Phone: +1 (915) 209-5153